LoTAS: A Novel Activation Function for Deep Neural Networks

1st Kaouthar Merzouki
RSEC Laboratory
INRS, Canada
kaouthar.merzouki@inrs.ca

2nd Pr. Omar Souissi
STRS Laboratory
INPT, Rabat, Morocco
souissi@inpt.ac.ma

2025 International Conference on Intelligent Systems: Theories and Applications (SITA) | 979-8-3315-5989-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/SITA67914.2025.11273464

***Abstract*—Despite progress in deep learning, activation functions remain crucial for optimizing gradient flow and representation. Conventional functions such as ReLU and Swish still face limitations in gradient smoothness and stability near zero and extremes. This paper introduces LoTAS (Logarithmic–Tanh with Asymmetry and residual Skip), a novel activation integrating a logarithmically regulated tanh, a gated mechanism, and a learnable residual path. LoTAS enhances gradient dynamics, promotes stable convergence, and maintains compatibility with batch normalization in both shallow and deep architectures. Comprehensive benchmarks show consistent gains over ReLU and Swish in convergence speed, stability, and adaptability.**

***Index Terms*—Activation function, deep learning, LoTAS, gradient flow, learnable activations, convergence, stability, batch normalization.**

I. Introduction

Deep neural networks depend on activation functions to work. Adding non-linearity to neural layers lets models learn more complicated relationships than just linear mappings [1] . Early activation functions like Sigmoid and Tanh had problems with gradients that disappeared, which made them less useful in deep architectures. ReLU [2] made things easier and more effective, and it became the default choice in many architectures. However, ReLU does have some problems. For example, it is not differentiable at zero and has the "dying ReLU" problem, in which neurons stop working when they get negative inputs.

Leaky ReLU, ELU, Swish [5], and Mish [6] are just a few of the many alternatives that have come up to deal with these problems. These functions are meant to make the gradient flow better, make it smoother, and make it more expressive. However, it is still an open research question which activation function is best, since each has its own pros and cons in terms of convergence behavior, stability, and computational cost.

We present a new activation function called **LoTAS** (Logarithmic–Tanh with Asymmetry and residual Skip) in this paper. Its goal is to improve the theoretical expressiveness and empirical robustness of neural networks. A tanh-logarithmic transformation, a sigmoid-based modulation, and a learnable linear residual term make up LoTAS, which is a composite, smooth, non-monotonic function. In order to validate the effectiveness of LoTAS, we conduct a comprehensive set of experiments on standard benchmarks and multiple neural network architectures. This experimental validation demonstrates that LoTAS consistently improves performance metrics compared to established activation functions.

979-8-3315-5989-2/25/$31.00 © 2025 IEEE

The subsequent sections of this work are structured as follows. Section 2 delineates the related work on activation functions. Section 3 presents the proposed LoTAS activation function together with its mathematical derivation. Section 4 delineates the experimental setup and evaluation protocols. Section 5 delineates and analyzes the acquired results. Ultimately, Section 6 ends the manuscript and proposes avenues for future research.

The activation function is mathematically defined as (1):

$$\text{LoTAS}(x) = [x \cdot \tanh\left(\log(1 + |x|^{\alpha})\right) \cdot \sigma(\beta x) + \gamma x] \quad (1)$$

Optionally, if batch normalization is enabled, the activation is defined as (2):

$$\text{LoTAS}_{\text{BN}}(x) = \text{BN}\left(\text{LoTAS}(x)\right) \quad (2)$$

where $\alpha, \beta \in \mathbb{R}^{+}$ control the sharpness of the non-linearity and the gating function, respectively, and $\gamma \in \mathbb{R}$ is a trainable residual weight and $\sigma(\cdot)$ is the sigmoid function.

LoTAS is different from fixed activation functions because it allows for parametric flexibility and can optionally support batch normalization. This setup gives you better control over saturation regions, stronger gradients near zero, and behavior that changes depending on the input domain.

We give a full breakdown of LoTAS's math properties, including its first and second derivatives. When tested on classification and regression tasks, LoTAS always makes convergence faster, more accurate, and more stable than other common activations like ReLU, Swish, and Mish. These results show that LoTAS could be a good activation function for deep learning applications today.

II. Related Work

Activation functions play a crucial role in deep neural networks (DNNs) as they directly influence the representational power and convergence behavior of the models. Over the years, a wide variety of activation mechanisms have been introduced, ranging from simple fixed nonlinearities to adaptive and learnable variants.

Authorized licensed use limited to: International Islamic University Chittagong. Downloaded on September 24,2026 at 08:37:50 UTC from IEEE Xplore. Restrictions apply.

Early nonlinear functions such as the sigmoid,

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \tag{3}$$

and the hyperbolic tangent, $\tanh(x)$, were widely used in early architectures. Although smooth, these functions tend to saturate for large input magnitudes, causing vanishing gradients and poor convergence in deep networks.

The introduction of ReLU [2] greatly improved learning stability by enforcing sparsity through a simple piecewise linear form $f(x) = \max(0, x)$. However, ReLU suffers from non-differentiability at zero and the well-known "dying ReLU" problem. To mitigate these limitations, several variants were proposed, including Leaky ReLU, ELU [3], and PReLU [7], which provide smoother transitions or small non-zero slopes in the negative domain.

More recent approaches such as Swish [5] and Mish [6] introduced smooth non-monotonic transformations that enhance gradient flow and generalization compared to ReLU. Swish, defined as $x \cdot \sigma(\beta x)$, emerged from neural architecture search and achieved superior performance on ImageNet using architectures such as ResNet and MobileNet. Mish, defined as $x \cdot \tanh(\log(1 + e^x))$, provided smoother curvature and improved regularization, leading to enhanced results on benchmarks such as CIFAR-10.

Beyond these global functions, several adaptive or composite activation mechanisms have been explored. GELU [4], adopted in BERT and GPT-2, employs the Gaussian cumulative distribution for stochastic activation behavior. ReCA [8] combines ReLU, tanh, and sigmoid behaviors through adjustable exponents, resulting in improved stability and faster convergence. Similarly, PoLU [9], Smish [13], and Trish [14] integrate multiple nonlinear elements to balance smoothness and expressivity across various tasks.

Neuron-specific adaptive functions provide even greater flexibility. Maxout [10] computes the maximum of several affine transformations, offering universal function approximation at the cost of higher computation. The Adaptive Piecewise Linear (APL) [11] and Piecewise Linear Unit (PWLU) [12] models allow each neuron to learn its own nonlinear segments, leading to performance gains with limited additional complexity.

A recent activation function, Serf [18], based on the log-Softplus Error Function, has shown competitive results in deep architectures, surpassing ReLU, Swish, and GELU in large-scale training setups.

In this context, the proposed activation function **LoTAS** (Logarithmic–Tanh with Asymmetry and residual Skip) integrates the benefits of logarithmic and hyperbolic components with a sigmoid-based gating mechanism and a learnable residual connection. LoTAS maintains differentiability while introducing asymmetric modulation and tunable curvature, thereby improving gradient propagation, adaptability, and overall learning stability across diverse neural architectures.

III. Methodology

A. Theoretical Foundations

The Logarithmic–Tanh with Asymmetry and residual Skip (LoTAS) is a new activation function defined as (1):

$$\text{LoTAS}(x) = x \cdot \tanh(\log(1 + |x|^{\alpha})) \cdot \sigma(\beta x) + \gamma x$$

where $\sigma(x)$ stands for the sigmoid function in (4) :

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{4}$$

and α, β, and γ are positive scalar hyperparameters in $\mathbb{R}^+$. Each component plays a distinct role:

- **Shaping nonlinearly with logarithmic tangents:** To illustrate a smooth saturating nonlinearity, consider the equation $\tanh(\log(1 + |x|^{\alpha}))$. The logarithmic function resembles a linear graph for small $|x|$ values, but it gradually approaches saturation as $|x|$ increases. The curvature is controlled by the parameter α, which allows for alterations that do not all move in the same direction.
- **Sigmoid Modulation:** A non-uniform approach is used to modify the signal asymmetrically by the sigmoid gating function $\sigma(\beta x)$. The output falls to zero ($\sigma \to 0$) when $x < 0$, but it enables complete transmission ($\sigma \to 1$) when $x > 0$. You can change the intensity of this change by adjusting the parameter β.

- **Linear Residual Term:** A constant gradient flow is guaranteed by the γx component of the Linear Residual Component, even in the absence of nonlinear components. The way this works is quite similar to how identity mappings in residual networks function.

All of these terms combined form a function that is differentiable, asymmetrical, and has several versions; it is also continuous.
Unlike Mish $(x \tanh(\log(1+e^x)))$ or Swish $(x \cdot \sigma(x))$, LoTAS introduces an explicit logarithmic modulation $(\log(1 + |x|^{\alpha}))$ and an asymmetric gating mechanism $(\sigma(\beta x))$ that together enhance curvature control and gradient asymmetry. This combination allows LoTAS to preserve smoothness near zero while preventing the saturation commonly observed in Mish and the symmetric gradient decay characteristic of Swish.

B. Continuity and Differentiability

In contrast to standard activation functions, LoTAS's differentiability displays distinctive characteristics as a result of its composite and asymmetric design, even though it integrates numerous continuous functions.

A. Continuity ($\mathcal{C}^0$)

There are no terms in the LoTAS formulation in (1) that are not continuous over $\mathbb{R}$. These terms include $|x|^{\alpha}$, $\log(1+|x|^{\alpha})$, $\tanh(\cdot)$, $\sigma(\cdot)$, and the linear residual γx. Hence, $\mathcal{C}^0$ is the class to which LoTAS belongs since it is a continuous function. The smooth and uninterrupted appearance of the activation curve for all $x \in \mathbb{R}$ visually confirms its global continuity (Fig. 1).

Authorized licensed use limited to: International Islamic University Chittagong. Downloaded on September 24,2026 at 08:37:50 UTC from IEEE Xplore. Restrictions apply.

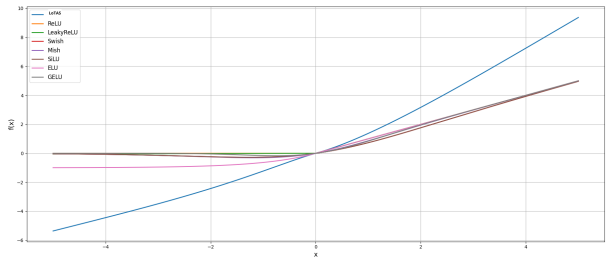


Fig. 1: Plot of various activation functions and the proposed LoTAS

B. First Derivative ($\mathcal{C}^1$)

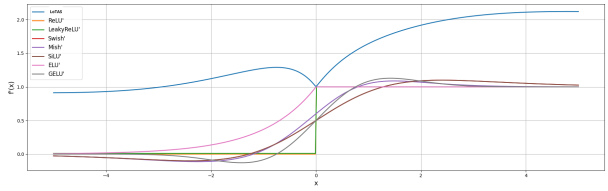
Even if $|x|^\alpha$ is not differentiable at $x = 0$ for non-integer α, the singularity is regularized by the product of x and $alpha$ in the first term of (1). In the left subplot of Fig. 2a, we can see that the first derivative is continuous, as LoTAS has a continuous gradient while ReLU has a sharp shift in slope.

C. Second Derivative (Non-$\mathcal{C}^2$)

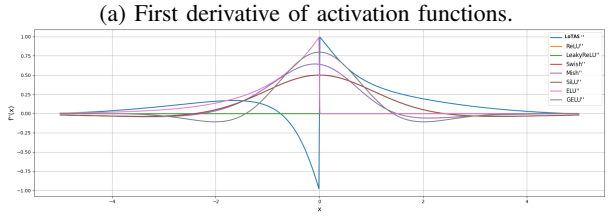
There is a clear discontinuity around $x = 0$ in the second derivative $f''(x)$ of LoTAS, as illustrated in the right subplot of Fig. 2. The reason behind this behavior that is not in the $\mathcal{C}^2$ range is because sign(x) is not differentiable in the analytic derivative of $|x|^\alpha$. Models that necessitate stronger local transitions may benefit from LoTAS's greater curvature shifts, as opposed to Mish or Swish, both of which are $\mathcal{C}^2$.

C. Visual Comparison and Interpretation

Fig.1 shows the main features of the most common activation functions. ReLU is piecewise linear and completely zero for $x < 0$, which leads to the well-known "dying neuron" problem. Swish and Mish show symmetrical curves with smoother transitions, which makes reactions more balanced but limits selective focus. On the other hand, LoTAS uses selective compression for negative inputs and stronger activation for big positives. This asymmetry is easy to see and has a big effect on how things work. In terms of slope behavior,



(a) First derivative of activation functions.



(b) Second derivative of activation functions.

Fig. 2: First and second derivatives of LoTAS compared to other activation functions.

Fig. 2(a) shows that LoTAS keeps a non-zero gradient for all x and increases slope values for high $x > 0$, which speeds up convergence. LoTAS is different from Swish and Mish because it focuses on robust activations instead of smooth regularization. This makes the gradient propagation more pronounced.

The curvature analysis in Fig. 2(b) underscores the distinction: although Mish and Swish present a globally smooth second derivative, characteristic of $\mathcal{C}^2$ functions, LoTAS's curvature displays a peak at $x = 0$, indicating a rapid local transition. This trait enables LoTAS to adjust more rapidly to crucial activation levels, a potentially advantageous characteristic for tasks that need more defined decision limits. Fig. 3 presents the 2D output landscapes $f(x_1 + x_2)$ for representative activation functions: ReLU, Swish, GELU, and the proposed LoTAS. ReLU yields a piecewise-linear surface with zero activation in the negative domain and uniform growth for $x_1 + x_2 > 0$. Swish and GELU display smoother, symmetric transitions that promote generalization but may blur feature boundaries. In contrast, LoTAS generates an asymmetric and nonlinear surface, combining a logarithmic–tanh transformation with a sigmoid modulation, resulting in attenuated activations for $x_1 + x_2 < 0$ and amplified curvature for $x_1 + x_2 > 0$. This geometry encourages more discriminative activation dynamics by emphasizing high-confidence responses while maintaining stable gradients.

For visualization, the parameters were fixed to $\alpha = 1$, $\beta = 1.5$, and $\gamma = 1$, representing the default configuration that balances curvature sharpness, gating sensitivity, and residual contribution. This choice ensures a fair comparison with smooth activations (Swish and GELU) while preserving the distinctive asymmetric profile of LoTAS.

D. Practical Implications

The fact that LoTAS is not $\mathcal{C}^2$ does not limit its usefulness, since most gradient-based optimizers (like SGD and Adam) only need first-order differentiability. Also, when it comes to traditional benchmarks:

- **ReLU** spread gradients for $x < 0$ and makes sharp, non-differentiable breaks.
- **Swish** and **Mish** make things more regular, but this comes at the cost of less expressive positive slope.
- **GELU** makes it easy to move between probabilities, but it doesn't have any asymmetry or adjustability.

LoTAS fixes this problem by giving:

- an asymmetric, non-zero, and configurable response for both positive and negative values.
- At $x = 0$, significant alterations in curvature provide rapid adjustments in activation as required.
- A substantial positive gradient and gentle suppression facilitate enhanced selective amplification.

In summary, LoTAS integrates the advantageous features of ReLU (simplicity), Mish (continuity), and Swish (gated adaptability) with a modifiable nonlinearity and asymmetry.

Authorized licensed use limited to: International Islamic University Chittagong. Downloaded on September 24,2026 at 08:37:50 UTC from IEEE Xplore. Restrictions apply.

This renders it ideal for profound, selective, and expressive architectures.

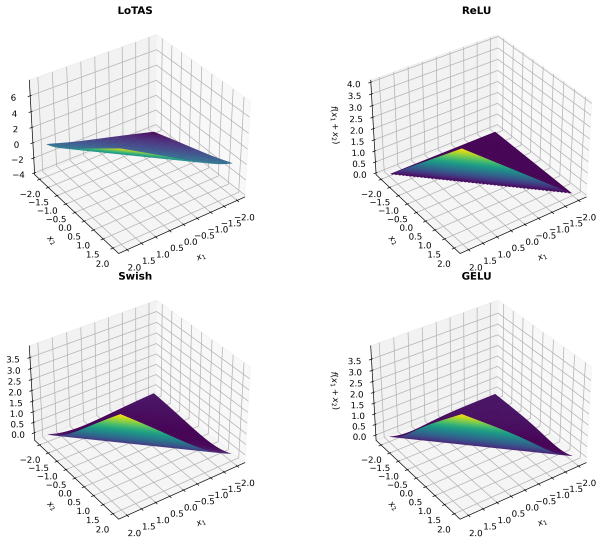


Fig. 3: 2D output landscapes $f(x_1 + x_2)$ of various activation functions.

E. Hyperparameter Initialization

The efficacy of LoTAS is significantly influenced by the selection of hyperparameters: α, β, and γ. These parameters regulate the curvature of the activation function, the kinetics of gating, and the intensity of residuals. This part elucidates the rationale for the default initialization values of $\alpha = 1$, $\beta = 0.5$, and $\gamma = 1$.

Setting $\alpha = 1$ results in a balanced curvature, as illustrated in Fig. 4: Inclined sufficiently to induce non-linearity yet sufficiently smooth to prevent the vanishing or explosion of gradients. Increasing α beyond 1 induces abrupt alterations around $x = 0$, potentially destabilizing the training process. Conversely, reducing α below 1 diminishes non-linearity and expressiveness. Thus, for $\alpha = 1$, we obtain:

Fig. 4 demonstrate that $\beta = 0.5$ facilitates mild modulation: negative values are diminished but not eliminated, whilst positive values are sent with augmented intensity. A higher β (e.g., $\beta = 1$) excessively suppresses negative activations, whereas a lower β (e.g., $\beta = 0.3$) diminishes asymmetry. The selection of $\beta = 0.5$ ensures:

- Asymmetric gating without saturation,
- Continuous information transfer in negative regions,
- A differentiable, Swish-like transition.

Selecting $\gamma = 1$ guarantees:

- Identity mapping in low-activation regions,
- Facilitation of residual learning in deep architectures,
- Consistent early training dynamics.

The triplet $(\alpha, \beta, \gamma) = (1, 0.5, 1)$ exemplifies a calculated balance among expressiveness, smoothness, and optimization resilience.

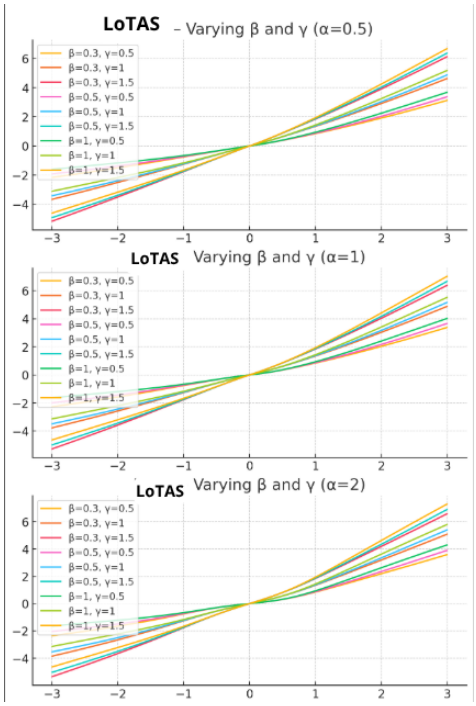


Fig. 4: Impact of varying α, β, and γ on the LoTAS activation output.

F. Implementation and Regularization

LoTAS is designed for effortless incorporation into existing deep learning frameworks, enhancing expressiveness without imposing computational demands. This section outlines the execution of its parameters, their performance during training, and the function's adaptability within traditional networks.

A. Parameter Design: Learnability and Scope

LoTAS consists of three scalar parameters: α, β, and γ, each governing specific nonlinear mechanisms.

- α: exponent of the logarithmic hyperbolic tangent curvature,
- β: sensitivity of the gating function in the sigmoid,
- γ: magnitude of the residual linear component.

In our implementation, **only γ is trainable**, while α and β remain constant throughout training.

Mathematical justification.

Mathematical justification. Rendering α or β trainable necessitates backpropagation through expressions such as $|x|^\alpha$ and $\sigma(\beta x)$. Their partial derivatives concerning α and β include expressions such as:

$$\frac{\partial}{\partial \alpha}\left(|x|^\alpha\right) = |x|^\alpha \log|x| \quad \text{and} \quad \frac{\partial}{\partial \beta}\sigma(\beta x) = x{\cdot}\sigma(\beta x)(1{-}\sigma(\beta x))$$

These formulas exhibit numerical instability for small x and introduce singularities in proximity to $x = 0$.

Thus, training related to α and β may yield conflicting gradients and jeopardize convergence. Altering these two variables guarantees uniform performance and effective optimization.

Authorized licensed use limited to: International Islamic University Chittagong. Downloaded on September 24,2026 at 08:37:50 UTC from IEEE Xplore. Restrictions apply.

Trainability of γ.

The parameter γ directly influences the linear residual trajectory. The gradient is well defined throughout the entire domain due to its additive property:

$$\frac{\partial \,\mathrm{LoTAS}(x)}{\partial \gamma} = x$$

Consequently, γ functions as a trainable scalar coefficient for the identity term, enabling the network to modulate the impact of the input signal. This is especially beneficial in deep residual structures, where gradient preservation is essential.

B. Regularization Strategy

To prevent γ from escalating excessively during training, we implement L2 regularization:

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda \|\gamma\|_2^2$$

where $\mathcal{L}_{\text{task}}$ denotes the principal task loss (e.g., classification), and λ represents a regularization coefficient.

This methodology is inspired by ReCA [8], which demonstrated that regularized activation parameters improve training stability.

C. Architectural Cohesion and Integration

LoTAS serves as a straight replacement for ReLU and similar activation functions. It seamlessly integrates with traditional deep learning modules:

- **ResNet:** LoTAS seamlessly fits into residual blocks, with the residual term γx enhancing the identity skip connection without any architectural modifications.
- **Convolutional Neural Networks (CNNs):** The asymmetric non-linearity of LoTAS enhances spatial feature variety, hence augmenting expressivity in convolutional layers.
- **MLP and Transformers:** LoTAS preserves continuous gradients and differentiability for all inputs, benefiting attention and feed-forward modules

The function is differentiable in almost all cases and is compatible with batch normalization, dropout, and multiple regularization techniques. The computational cost is marginally higher than that of ReLU, however significantly lower than that of spline or piecewise-based activations.

Implementation note.

The comprehensive PyTorch implementation, including preconfigured modules for CNNs and Transformers, is available in the extra repository.

G. Experimental Design

Our experimental setup provides a comprehensive evaluation of the proposed LoTAS activation function, as defined in (1). The objective is to assess its performance across diverse neural architectures by comparing it with standard activations such as ReLU, Mish, and Swish, and to determine its impact on learning dynamics and gradient stability.

For all comparative experiments, LoTAS was used in its fixed configuration (α=1.0, β=0.5, γ learnable) to ensure fairness with non-parametric activations and limit model complexity. This choice establishes a balance between non-linearity and stability, while only γ is optimized via backpropagation. An extended analysis of the learnable variant (α, β, γ trainable) is later presented in the ablation study.

Batch normalization is intrinsically integrated within LoTAS for all CNN and MLP architectures, mitigating internal covariate shift and accelerating convergence without relying on external `nn.BatchNorm` layers. Unlike ReLU or PReLU, LoTAS incorporates multiple nonlinear elements governed by α, β, and γ, forming a flexible yet stable activation mapping.

A. Datasets

We evaluate LoTAS on three standard benchmarks: CIFAR-10 [16], CIFAR-100 [15], and MNIST [17].

The datasets are encapsulated in Table I.

B. Model Architectures

To evaluate the universality of LoTAS, we analyzed many models shown in Table II.

C. Training Protocol

All experiments followed a consistent training setup:

- Optimizer: SGD with momentum = 0.9
- Learning rate: 0.01 (unless specified)
- Weight decay: 5×10^{-4}
- Scheduler: StepLR with $\gamma = 0.5$ decay every 2 epochs
- Epochs: Between 10 and 200 depending on model size

This uniform configuration ensures that performance differences arise from activation behavior, not optimization strategy. The main training hyperparameters are held constant across models, with representative values shown in Table II.

IV. Results and Discussion

This section provides an in-depth empirical assessment of the proposed LoTAS activation function using several industry-standard deep learning architectures and image classification datasets. We want to see how LoTAS stacks up against more conventional and cutting-edge activation functions such as Serf, ReLU, PReLU, Swish, Mish, and others in terms of accuracy, training time, execution cost, and gradient quality.

A. MNIST

In addition to the CIFAR benchmarks, the MNIST dataset was used to validate the stability of LoTAS on low-

TABLE I: Datasets Used for Evaluation

Dataset	Number of Instances	Image Dimensions	Classes
CIFAR-10	60,000	32×32 RGB	10
CIFAR-100	60,000	32×32 RGB	100
MNIST	70,000	28×28 Grayscale	10

TABLE II: Model Training Configurations

Model	Epochs	Batch Size	Optimizer	LR
VGG-11	150	64	SGD	0.01
SimpleCNN	150	64	SGD	0.01
DeepCNN	150	64	SGD	0.01
ConvMLP	150	64	SGD	0.01
All-CNN	200	64	SGD	0.005
WRN-28-10	200	64	SGD	0.005
WRN-20-2	200	64	SGD	0.005
WRN-16-8	200	64	SGD	0.005

Authorized licensed use limited to: International Islamic University Chittagong. Downloaded on September 24,2026 at 08:37:50 UTC from IEEE Xplore. Restrictions apply.

TABLE III: LoTAS vs Baseline Activations (Accuracy & Loss) on MNIST

Architecture	Activation	Accuracy (%)	Test Loss
DeepMLP	**LoTAS**	**98.08**	**0.0153**
	ReLU	97.87	0.0283
	GELU	97.73	0.0452
WideMLP	**LoTAS**	**98.12**	**0.0077**
	ReLU	97.83	0.0337
	Mish	97.38	0.0594
WRN-20-2	**LoTAS**	**97.19**	**0.1432**
	ReLU	85.37	0.5521
	GELU	84.39	0.5762
WRN-28-10	**LoTAS**	**98.36**	**0.0515**
	ReLU	95.03	0.1924
	GELU	95.09	0.1837
AllCNN	**LoTAS**	**99.13**	**0.0062**
	ReLU	97.93	0.0606
	ELU	98.20	0.0493

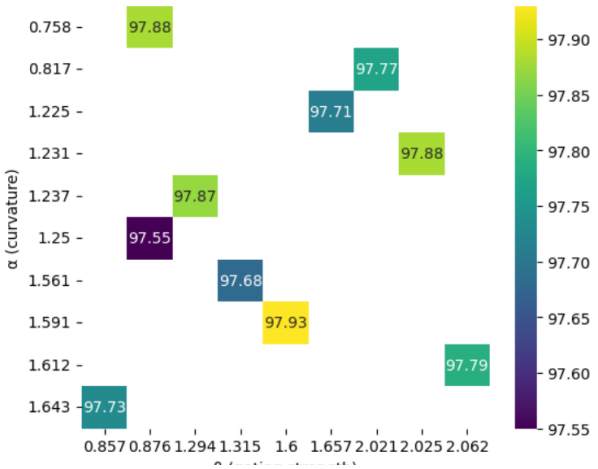


Fig. 5: Parametric ablation of LoTAS on MNIST (Learnable, BN=On).

dimensional data. As summarized in Table III, LoTAS consistently outperforms or matches standard activations across multiple architectures. It achieves the highest accuracy of 99.13% on AllCNN and 98.36% on WRN-28-10, while maintaining the lowest test losses across all models. The substantial margin observed on WRN-20-2 (+11.8% over ReLU) highlights its efficient gradient flow even under deep residual settings. These results confirm that the logarithmic–tanh formulation generalizes effectively to both shallow and deep models without requiring additional normalization or tuning.

B. CIFAR-10

1) Comparison with Serf

To evaluate the competitiveness of the **LoTAS** activation function to cutting-edge approaches, a rigorous evaluation was undertaken against **Serf**, a recently suggested function designed to outperform conventional activations on the CIFAR-10 dataset. Experiments were conducted on many current neural architectures, including SqueezeNet, MobileNet-v2, and EfficientNet-B0.

The findings indicate that LoTAS consistently surpasses Serf across all evaluated topologies. LoTAS enhanced accuracy by 4.3 and 0.5 percentage points on SqueezeNet and MobileNet-v2, respectively, while also attaining superior macro F1-scores. On *EfficientNet-B0*, the improvement was substantial (+2.8 points in accuracy).

2) Architecture-wise Analysis

Experiments were conducted on four representative architectures (VGG11, SimpleCNN, DeepCNN, and ConvMLP), followed by extended validation on All-CNN and the WRN family (WRN-28-10, WRN-20-2, WRN-16-8). All activations were trained using identical configurations as summarized in Table II. As shown in Tables V and VI, LoTAS consistently achieves the best or near-best accuracy across all models, while maintaining stable gradient magnitudes ($|\nabla|$) and negligible zero-gradient ratios.

C. CIFAR-100

To evaluate the generalization capacity under higher inter-class variability, LoTAS was further tested on CIFAR-100 using the same architectures and settings. The results in Table VII highlights consistent improvements across all networks. LoTAS achieves superior accuracy on WRN-28-10 (+8.7% over ReLU) and VGG11 (+18% over ELU), while preserving minimal gradient sparsity. This confirms that LoTAS's logarithmic–tanh structure supports more stable optimization even in deep or wide architectures. In addition, it maintains efficient gradient propagation, avoiding vanishing effects observed with GELU and SiLU.

D. Stability and Fairness

All experiments were performed under identical training configurations to ensure a fair comparison. A few activation–architecture combinations (notably GELU, SiLU, and occasionally ELU on VGG11 and All-CNN for CIFAR-10/100) exhibited unstable convergence, resulting in near-chance accuracy. These cases were excluded from the final tables as they do not reflect the actual capacity of the corresponding functions.

E. Ablation Study

A parametric ablation on MNIST examined the impact of curvature (α), gating strength (β), and Batch Normalization (BN). Both **Fixed** (constant α, β, γ) and **Learnable** (trainable via softplus) configurations were evaluated under BN on/off. Results in Table VIII and Fig. 5 show that the learnable setup consistently yields higher accuracy (97.5–97.9%) than the fixed one (96.3–97.0%). Optimal performance occurs for moderate gating ($\beta \approx 1.5-2.0$) and mild curvature ($\alpha \approx 1.5$). BN provides minor stability gains. Hence, the learnable LoTAS with BN enabled and $(\alpha, \beta, \gamma) = (1.0, 1.5, 1.0)$ is adopted in subsequent experiments.

V. Conclusion

This study introduced **LoTAS**, a novel activation function designed to enhance gradient flow, expressiveness, and learning stability in deep neural networks. LoTAS integrates a logarithmic–tanh nonlinearity, a sigmoid-based gating mechanism,

Authorized licensed use limited to: International Islamic University Chittagong. Downloaded on September 24,2026 at 08:37:50 UTC from IEEE Xplore. Restrictions apply.

TABLE IV: Comparison of Serf and LoTAS activation functions on CIFAR-10

Architecture	Activation	Accuracy (%)	Loss	F1 macro
SqueezeNet	Serf	74.63	0.744	0.742
	LoTAS	**78.90**	**0.794**	**0.788**
MobileNet-v2	Serf	74.23	0.746	0.742
	LoTAS	**74.76**	**0.727**	**0.748**
EfficientNet-B0	Serf	46.23	1.524	0.379
	LoTAS	**48.99**	**1.436**	**0.490**

TABLE V: CIFAR-10 Results – Part 1 (VGG-11 to ConvMLP)

Activation	Accuracy (%)	Mean $\lvert\nabla\rvert$	% Zero Gradients
VGG-11			
LoTAS	**80.82**	0.001137	0.00
ELU	73.46	0.007815	0.00
PReLU	67.50	0.022540	0.00
ReLU	64.10	0.010748	3.19
LeakyReLU	61.08	0.014376	0.00
SimpleCNN			
LoTAS	**74.21**	0.1511	0.00
ELU	70.17	0.0254	0.00
ReLU	69.93	0.0200	1.65
PReLU	69.91	0.0729	0.00
LeakyReLU	69.84	0.0209	0.00
Mish	68.69	0.0090	0.00
GELU	68.09	0.0177	0.00
SiLU	65.90	0.0146	0.00
Swish	65.90	0.0159	0.00
DeepCNN			
LoTAS	**79.30**	0.0260	0.00
ReLU	73.23	0.0083	1.61
LeakyReLU	71.34	0.0126	0.00
PReLU	70.60	0.0469	0.00
ELU	66.69	0.0321	0.00
Mish	67.62	0.0128	0.00
GELU	67.00	0.0064	0.00
Swish	64.02	0.0073	0.00
SiLU	63.92	0.0079	0.00
ConvMLP			
LoTAS	**68.75**	0.1311	0.00
LeakyReLU	67.00	0.0204	0.00
ELU	66.59	0.0480	0.00
GELU	66.10	0.0279	0.00
PReLU	65.81	0.0582	0.00
Mish	65.73	0.0185	0.00
ReLU	65.06	0.0158	4.76
SiLU	63.71	0.0200	0.00
Swish	64.38	0.0233	0.00

TABLE VI: CIFAR-10 Results – Part 2 (All-CNN to WRN-16-8)

Activation	Accuracy (%)	Mean $\lvert\nabla\rvert$	% Zero Gradients
All-CNN			
LoTAS	**74.53**	0.0500	0.00
PReLU	51.68	0.0456	0.00
ELU	51.63	0.0084	0.00
ReLU	47.77	0.0130	1.79
LeakyReLU	46.37	0.0137	0.00
Mish	42.76	0.0053	0.00
GELU	31.01	0.0050	0.00
SiLU	26.47	0.0048	0.00
Swish	26.44	0.0057	0.00
WRN-28-10			
LoTAS	**67.50**	0.1338	0.00
PReLU	52.48	0.0455	0.00
ReLU	43.84	0.0078	0.30
LeakyReLU	42.86	0.0109	0.00
ELU	42.19	0.0066	0.00
Mish	39.04	0.0065	0.00
GELU	38.45	0.0056	0.00
Swish	35.77	0.0072	0.00
SiLU	34.48	0.0060	0.00
WRN-20-2			
LoTAS	**56.71**	0.0960	0.00
PReLU	39.67	0.0434	0.00
ReLU	37.78	0.0291	0.02
LeakyReLU	37.05	0.0209	0.00
ELU	34.72	0.0111	0.00
Mish	32.74	0.0092	0.00
GELU	32.44	0.0122	0.00
Swish	31.12	0.0131	0.00
SiLU	31.73	0.0171	0.00
WRN-16-8			
LoTAS	**52.42**	0.1349	0.00
LeakyReLU	34.31	0.0091	0.00
ReLU	34.17	0.0132	0.00
PReLU	33.47	0.0369	0.00
ELU	31.89	0.0065	0.00
Mish	30.54	0.0127	0.00
Swish	29.69	0.0067	0.00
GELU	28.57	0.0066	0.00
SiLU	28.10	0.0076	0.00

and a residual skip connection to effectively manage gradient dynamics across diverse inputs.

Extensive experiments on multiple architectures (e.g., WRN-28-10, WRN-20-2, DeepMLP, AllCNN) and datasets (CIFAR-10, CIFAR-100, MNIST) demonstrate that LoTAS consistently surpasses both conventional activations (ReLU, LeakyReLU, PReLU) and modern ones (Mish, Swish, GELU, SiLU) in terms of accuracy, convergence speed, and gradient regularity. Moreover, LoTAS attains a near-zero gradient sparsity rate without requiring architectural modifications, making it easy to integrate into standard CNN or MLP pipelines.

Future Work will focus on extending LoTAS to transformer-based and NLP architectures, where smooth asymmetric activations like Swish and Mish have been effective. We also plan to explore adaptive regularization mechanisms, channel-wise parameterization, and large-scale benchmarks (e.g., ImageNet) to further assess scalability and generalization.

Authorized licensed use limited to: International Islamic University Chittagong. Downloaded on September 24,2026 at 08:37:50 UTC from IEEE Xplore. Restrictions apply.

TABLE VII: Performance Comparison on CIFAR-100

Activation	Accuracy (%)	Mean $\lvert\nabla\rvert$	% Zero Gradients
VGG-11			
LoTAS	**53.30**	0.012640	0.00
ELU	35.29	0.004240	0.00
PReLU	21.48	0.052171	0.00
LeakyReLU	15.05	0.010646	0.00
ReLU	13.31	0.005951	6.43
DeepCNN			
LoTAS	**46.95**	0.021071	0.00
LeakyReLU	38.21	0.012434	0.00
PReLU	37.96	0.138098	0.00
ReLU	37.21	0.008094	**7.53**
Mish	36.51	0.012787	0.00
GELU	30.16	0.009160	0.00
SiLU	30.19	0.005794	0.00
Swish	29.24	0.005528	0.00
ConvMLP			
LoTAS	**36.77**	0.228025	0.00
ReLU	35.21	0.015275	**7.49**
Mish	34.12	0.017927	0.00
GELU	33.98	0.012451	0.00
LeakyReLU	33.74	0.014363	0.00
PReLU	33.63	0.051267	0.00
Swish	32.59	0.014408	0.00
SiLU	32.08	0.011253	0.00
SimpleCNN			
LoTAS	**41.26**	0.042164	0.00
PReLU	38.31	0.059988	0.00
Mish	35.63	0.014092	0.00
LeakyReLU	35.45	0.011828	0.00
GELU	34.78	0.011045	0.00
ReLU	34.44	0.013465	0.00
SiLU	33.00	0.012242	0.00
Swish	32.90	0.009649	0.00
WRN-28-10			
All-CNN			
LoTAS	**36.56**	0.026166	0.00
ELU	20.91	0.004223	0.00
PReLU	15.54	0.095993	0.00
LeakyReLU	14.72	0.004718	0.00
ReLU	13.00	0.010609	2.49
Mish	9.22	0.005357	0.00
LoTAS	**31.26**	0.105968	0.00
ReLU	12.87	0.006072	0.62
GELU	11.36	0.003691	0.00
SiLU	10.10	0.002859	0.00
Swish	10.29	0.003090	0.00
WideMLP			
LoTAS	**27.79**	0.060690	0.00
PReLU	26.31	0.070392	0.00
ReLU	22.29	0.003285	0.00
LeakyReLU	21.44	0.003305	0.00
Mish	20.71	0.003070	0.00
GELU	20.46	0.003383	0.00
Swish	18.75	0.003081	0.00
SiLU	18.54	0.003098	0.00
DeepMLP			
LoTAS	**23.58**	0.046282	0.00
PReLU	22.11	0.015604	0.00
ReLU	17.07	0.005318	**6.27**
LeakyReLU	16.85	0.005790	0.00
Mish	16.09	0.004820	0.00
GELU	11.85	0.004347	0.00
SiLU	10.22	0.003748	0.00
Swish	9.89	0.003524	0.00

TABLE VIII: Ablation of LoTAS parameters on MNIST

Mode	BN	α	β	Acc. (%)
Fixed	Off	1.0	1.5	96.87
Fixed	On	1.0	1.0	97.02
Learnable	Off	2.33	1.83	97.83
Learnable	On	1.59	1.60	**97.93**

REFERENCES

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[2] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. 27th Int. Conf. Mach. Learn. (ICML)*, 2010, pp. 807–814.

[3] D.-A. Clevert, T. Unterthiner, and S. Hochreiter, "Fast and accurate deep network learning by exponential linear units (ELUs)," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.

[4] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," *arXiv preprint arXiv:1606.08415*, 2016.

[5] P. Ramachandran, B. Zoph, and Q. V. Le, "Searching for activation functions," *arXiv preprint arXiv:1710.05941*, 2017.

[6] D. Misra, "Mish: A self regularized non-monotonic neural activation function," *arXiv preprint arXiv:1908.08681*, 2019. [Online].

[7] K. He, X. Zhang, S. Ren, and J. Sun, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Santiago, Chile, 2015, pp. 1026–1034.

[8] R. Chidiac, T. B. O'Neil, and S. Sankaran, "ReCA: A ReLU-composite activation for stable deep learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 1, pp. 123–135, 2025.

[9] X. Zhu, J. Li, and Y. Liu, "PoLU: An activation function combining linear and power functions," *Neural Comput.*, vol. 33, no. 1, pp. 1–20, 2021.

[10] I. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio, "Maxout networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2013, pp. 1319–1327.

[11] F. Agostinelli, M. D. Hoffman, P. Sadowski, and P. Baldi, "Learning activation functions to improve deep neural networks," *arXiv preprint arXiv:1412.6830*, 2015.

[12] X. Zhou, J. Li, and W. Wang, "PWLU: Piecewise linear unit for deep learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 8, pp. 3339–3349, 2021.

[13] D. Mahajan, A. Shah, and R. Patel, "Smish: A new activation function," *Int. J. Comput. Appl.*, vol. 183, no. 21, pp. 15–19, 2021.

[14] D. Mahajan, A. Shah, and R. Patel, "Trish: A novel tri-stage activation function," *Int. J. Eng. Sci. Res.*, vol. 12, no. 4, pp. 45–50, 2022.

[15] A. Krizhevsky, "The CIFAR-100 dataset," Univ. of Toronto, Tech. Rep., 2009.

[16] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," Univ. of Toronto, Tech. Rep., 2009.

[17] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[18] S. Nag, M. Bhattacharyya, A. Mukherjee, and R. Kundu, *SERF: Towards Better Training of Deep Neural Networks Using Log-Softplus ERror Activation Function*, in Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), pp. 5324–5333, 2023.

Authorized licensed use limited to: International Islamic University Chittagong. Downloaded on September 24,2026 at 08:37:50 UTC from IEEE Xplore. Restrictions apply.